T5. Iluminación

Luz

La luz es una **onda electromagnética** que es percibida por el ser humano y cuya frecuencia o suma de frecuencias determina su color. Se caracteriza por tres propiedades físicas:

- **Frecuencia** y/o **longitud de Onda**: son inversamente proporcionales y definen el **color** de la luz.
- **Intensidad** (Amplitud): determina el brillo que transporta una onda.
- **Pureza** o **saturación** (Espectro): conjunto de las frecuencias que se perciben.

- <u>Caracterización Humana de la Luz</u>

En el ojo humano hay dos tipos fundamentales de detectores:

- **Bastones**: células fotorreceptoras especializas en la **visión nocturna** que responden a la luminosidad, por lo que **no distinguen colores**. Tienen su máxima sensibilidad en la luz verde.

- **Conos**: células fotorreceptoras responsables de la **visión diurna** y la **percepción del color**. Hay 3 tipos, cada uno sensible a distintas longitudes de onda: **rojos** o conos-L, longitudes de onda **largas**; **verdes** o conos-M, longitudes de onda **medias**, y **azules** o conos-S, longitudes de onda **cortas**.

El espectro de la intensidad luminosa, en función de la longitud de onda λ es:

$$I(\lambda) = I_r(\lambda) + I_v(\lambda) + I_A(\lambda)$$

Por esta razón, en las **imágenes de síntesis** se caracteriza un color por medio de sus componentes RGB.

- <u>Teoría del Color: Colores Aditivos</u>

La teoría de los **colores aditivos** se basa en la combinación de luz para crear diferentes colores. En esta teoría los tres colores primarios son rojo, verde y azul. La **suma** de estos colores permite crear todos los demás.

La mezcla de **dos colores primarios** en proporciones iguales produce colores **secundarios.** La mezcla de los **tres colores primarios** en intensidades máximas produce luz **blanca**. La **ausencia** total produce **negro.** OpenGL, permite especificar colores utilizando valores para los componentes rojo, verde y azul, en un rango de 0 a 1 ($glColor3f(R, G, B)$).

- <u>Interpolación del Color</u>

En Opengl, se **interpola** el color de las caras a partir del **color** de los **vértices** que las forman. Para obtener un color determinado, se dibujan 3 triangulos y la intensidad final es la suma de los tres.

Modelos de Iluminación

Hay dos grandes tipos de modelos de iluminación:

- **Locales**: se utilizan en aplicaciones donde prima la **velocidad de renderizado,** sólo consideran las **características inmediatas del punto** y su entorno cercano, sin tener en cuenta las interacciones complejas de la luz con otros objetos de la escena. El más clásico es el modelo de Phong.

- **Globales**: consideran la **escena en conjunto** y las **interacciones complejas** de la luz en toda la escena, incluyendo los efectos indirectos entre objetos. Se utilizan en aplicaciones donde prima la calidad, porque aunque son más lentos, los resultados son mejores.

- <u>**Modelo Local de Iluminación**</u>

Los cálculos se realizan en función de:

- **Fuentes de luz**, caracterizadas por la posición (localizada o infinitamente alejada), intensidad y color de la fuente, número de luces, distribución luminica (uniforme, focalizada, direccional...), geometría (puntual, esférica, lineal...).

- **Objetos**, caracterizados por las distancias (al observador y a la fuente de luz), material o propiedades ópticas (transparencia, refracción...), cromaticidad (color superficial propio), interacción (no existe interacción lumínica con otros objetos de la escena).

- **Observador**, caracterizado por la dirección de observación (cálculo de la intensidad).

- <u>**Modelo de Iluminación Simple**</u>

La iluminación en un punto de una escena puede representarse mediante la siguiente función:

$$I = f(p, pv, \{MGMO\}, \{MGCL\})$$

I es la intensidad luminosa observada en el punto p. p es el punto donde se calcula la iluminación. pv es la posición del punto de vista del observador. $\{MGMO\}$ es el modelo geométrico y material de los objetos. $\{MGCL\}$ es el modelo geométrico y cromático de la luz.

Para simplificar el modelo, se pueden asumir que las fuentes de luz son **puntuales** (pequeñas y redondas), los objetos son **opacos** y no emiten luz propia, y no existen **inter-reflexiones** (la luz no rebota de un objeto a otro), los objetos no generan **sombras entre sí**.

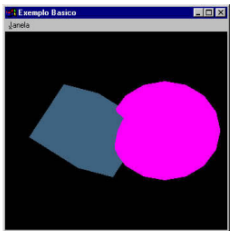
- <u>Modelo de Iluminación de Phong</u>

El modelo de phong es un **modelo empírico** de iluminación local caracterizado por usar fuentes de **luz puntuales** y por que los objetos (no fuentes de luz) **no emiten luz propia**. La luz está formada por:

- **Luz Ambiental:** es una luz **no direccional** (no provienen de una dirección concreta), no identificable. Su efecto solo depende de **su color** y del color del **objeto**. Su variación implica cambios uniformes en la iluminación de los objetos. No genera sombras ni matices de color. Incide sobre todas las partes del objeto por igual y muestra los objetos como siluetas:

$$I = I_a * K_a$$

  Donde: I_a es la **intensidad ambiental** en todo punto del espacio y K_a es el **coeficiente** de **reflexión** ambiental del objeto (entre 0 y 1), siendo 1 que el objeto refleja toda la luz ambiental.

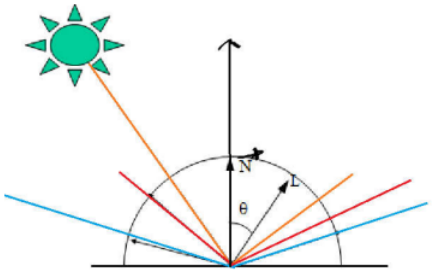


- **Luz Difusa** o **Luz Lambert**: está asociada a un **foco de luz**. Su variación implica **cambios de intensidad** o de posición del foco. Genera sombras y matices de color en función del ángulo con el que incide la luz en la superficie. Proviene de una **dirección concreta** y es reflejada en todas direcciones según la ley de Lambert, afectando aquellas partes del objeto en las que incide.

  Es reflejada por la **superficie** de un objeto de manera **uniforme** en todas direcciones, lo que significa que no depende del ángulo de observación, sino únicamente del **ángulo de incidencia** de la luz. Este comportamiento se rige por la **ley de Lambert**, que establece que la **intensidad** de la luz difusa reflejada es **proporcional** al **coseno** del ángulo entre la **dirección** de la luz **incidente** y la **normal** a la superficie.

$$I = I_L * K_d \cos(\theta) = I_L * K_d (\overline{N} * \overline{L})$$

Donde: I_L es la **intensidad** de la fuente de luz, K_d es el **coeficiente** de **reflexión** difusa del material [0, 1], θ es el **ángulo** de **incidencia** entre la normal y la dirección de la luz [0º, 90º], $\overline{N}$ es el **vector normal** a la superficie (unitario), $\overline{L}$ es el **vector de iluminación** (unitario).

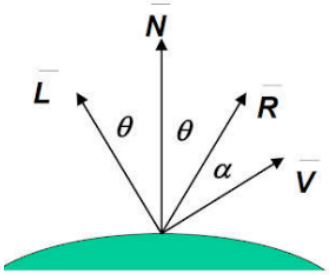


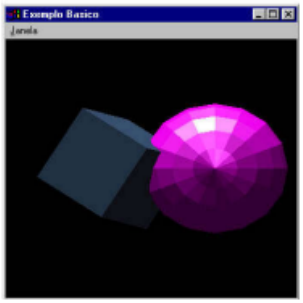
- **Luz Especular** o **Luz de Phong**: está asociada a un **foco intenso** de luz. Se usa en objetos con brillo, en caso extremo es un espejo. Su variación está asociada a **cambios de intensidad**, de **posición** del foco y de punto de **vista**. Proviene de una **dirección concreta** y depende de la posición del observador y delas propiedades del objeto. Está asociada a objetos **brillantes** o pulidos, además, las reflexiones especulares suelen del mismo color que la fuente de luz. En el caso ideal, la luz se refleja exactamente en el **ángulo incidente** (depende de este ángulo y de la posición del observador), produciendo un espejo.

  Superficies perfectas: el rayo indicente y el reflejado son coplanares.
  Superficies imperfectas: la luz se refleja dentro de un patrón tridimensional de reflexión, hay luz fuera del plano y el ángulo ideal de reflexión.

$$I = I_L * K_S \cos^n(\alpha) = I_L * K_S (\overline{R} * \overline{V})^n$$

Donde: I_L es la **intensidad** de la fuente de luz, K_S es el **coeficiente** de **reflexión** especular [0, 1] y depende del material del objeto, n es el **exponente** de brillo (1 es color mate e infinito es espejo), $\overline{R}$ es el **vector** de **reflexión** de la luz, y $\overline{V}$ es el **vector** que apunta al **observador**.





- **Luz Emisiva:** da apariencia de que el objeto **emite luz**.

- <u>Modelo Global de Iluminación Simple</u>

Modelo global de Iluminación simple es la suma de las tres iluminaciones (ambiental, difusa, especular):

$$I = I_a * K_a + I_L \left(K_d (\overline{N} * \overline{L}) + K_S (\overline{R} * \overline{V})^n \right)$$

Otros efectos:

- **Atenuación**: consiste en no desestimar la atenuación de la luz con la distancia al observador.

- **Múltiples fuentes**: para tener varias fuentes de luz en cuenta sólo hay que sumar sus efectos.

- **Color**: para tener en cuenta los tres colores hay que triplicar las expresiones.

$$I_R = I_{aR} \cdot K_a \cdot C_{dR} + I_{LR} \cdot f_{at} \cdot \left(K_d \cdot C_{dR} \cdot \left(\vec{N} \bullet \vec{L}\right) + K_S \cdot C_{eR} \cdot \left(\vec{R} \bullet \vec{V}\right)^n \right)$$

$$I_G = I_{aG} \cdot K_a \cdot C_{dG} + I_{LG} \cdot f_{at} \cdot \left(K_d \cdot C_{dG} \cdot \left(\vec{N} \bullet \vec{L}\right) + K_S \cdot C_{eG} \cdot \left(\vec{R} \bullet \vec{V}\right)^n \right)$$

$$I_B = I_{aB} \cdot K_a \cdot C_{dB} + I_{LB} \cdot f_{at} \cdot \left(K_d \cdot C_{dB} \cdot \left(\vec{N} \bullet \vec{L}\right) + K_S \cdot C_{eB} \cdot \left(\vec{R} \bullet \vec{V}\right)^n \right)$$

Modelos de sombreado (Shading)

- <u>Gouraud Shading</u>

El sombreado de Gouraud es una técnica de **rasterización** en la que la iluminación se calcula únicamente en los vértices de cada polígono. Para ello, primero se obtiene una **normal** media en cada vértice, y con ella se calcula el color en ese punto usando el modelo de iluminación. Una vez calculados los colores en los vértices, el **color** de cada píxel dentro del polígono se obtiene mediante **interpolación** (generalmente lineal o bilineal) entre esos **colores**.

- <u>Phong Shading</u>

El sombreado de Phong mejora significativamente la calidad visual. En lugar de interpolar colores, **interpola las normales** en cada punto del polígono y calcula la **iluminación por píxel**. Esto implica que cada fragmento tiene su propia normal, lo que permite obtener resultados mucho más realistas, especialmente en reflejos. No está implementado directamente en OpenGL 3.3 (se implementa manualmente en shaders)

Iluminación en OpenGL

En OpenGL, la iluminación se calcula en el **fragment shader**, después de que el vertex shader haya transformado los vértices.

- **Iluminación básica**

El **color final** de un objeto es el resultado de **multiplicar** el color del **objeto** por el color de la **luz**. Se definen dos variables que se pasan al shader.frag en el se incluye la multiplicación de cada uno de ellos.

```
version 330 core
uniform vec3 objectColor;
uniform vec3 lightColor;
void main() {
Gl_FragColor = vec4(lightColor * objectColor, 1.0); }
```

El **objectColor** es el color del objeto, **lightColor** es el color de la luz. Mientras que **vec4** añade el canal alfa 4.

- **Luz ambiental (Ambient Light)**

La luz ambiental simula la **iluminación indirecta** del entorno. No depende de la dirección ni de las normales.

```
//El color del objeto
unsigned int colorLoc = glGetUniformLocation(shaderProgram, "objectColor");
glUniform3f(colorLoc,  0.9f,0.3f,0.3f);
//El color de la luz
unsigned int lightLoc = glGetUniformLocation(shaderProgram, "lightColor");
glUniform3f(lightLoc, 0.9f, 0.6f, 0.6f);
```

```
uniform vec3 lightColor;
uniform vec3 objectColor;
void main()
{
    float ambientI = 0.1;
    vec3 ambient = ambientI * lightColor;
    vec3 result = ambient * objectColor;
    gl_FragColor = vec4(result, 1.0);
}
```

El **ambientI** es la intensidad de luz ambiental, **lightColor** es el color de la luz

- **Luz difusa (Diffuse Lighting)**

La luz difusa depende del ángulo entre la luz y la superficie (modelo de Lambert), se necesitan las normales de las caras, la atenuación, la posición de la luz y del punto de cálculo para ver la dirección de la luz.

1) Especificar las coordinadas de las normales.

```
float vertices[] = {
//Frontal vertices                          Normales
  -0.5f, -0.5f,  0.5f,              0.0f, 0.0f, 1.0f,
   0.5f, -0.5f,  0.5f,              0.0f, 0.0f, 1.0f,
   0.5f,  0.5f,  0.5f,              0.0f, 0.0f, 1.0f,
    0.5f,  0.5f,  0.5f,             0.0f, 0.0f, 1.0f,
   -0.5f,  0.5f,  0.5f,             0.0f, 0.0f, 1.0f,
   -0.5f, -0.5f, 0.5f,              0.0f, 0.0f, 1.0f,  …….
// vertices
glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 6 * sizeof(float), (void*)0);
glEnableVertexAttribArray(0);
// Normales
glVertexAttribPointer(1, 3, GL_FLOAT, GL_FALSE, 6 * sizeof(float), (void*)(3 *
sizeof(float)));
glEnableVertexAttribArray(1);
```

El atributo 0 es la posición y el atributo 1 es la normal. Cada vértice tiene 6 floats (3 posición + 3 normal).

```
#version 330 core
layout (location = 0) in vec3 aPos;
layout (location = 1) in vec3 aNormal;
out vec3 Normal;
out vec3 FragPos;
uniform mat4 model;
uniform mat4 view;
uniform mat4 projection;

void main()
{
   gl_Position = projection * view * model * vec4(aPos, 1.0f);
   Normal = mat3(transpose(inverse(model))) * aNormal;
   FragPos = vec3(model * vec4(aPos, 1.0));
}
```

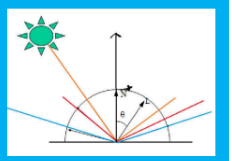
```
in vec3 Normal;
in vec3 FragPos;

uniform vec3 lightPos;
uniform vec3 lightColor;
uniform vec3 objectColor;
void main()
{
    // ambient
    float ambientI = 0.5;
    vec3 ambient = ambientI * lightColor;

    // diffuse
    vec3 norm = normalize(Normal);   //Normaliación
    vec3 lightDir = normalize(lightPos - FragPos);  //rayo luz fragmento
    float diff = max(dot(norm, lightDir), 0.0);     //producto escalar
    vec3 diffuse = diff * lightColor;

    vec3 result = (ambient + diffuse) * objectColor;
     FragColor = vec4(result, 1.0);

}
```



Normal es la matriz inversa transpuesta, **FragPos** es la posición del fragmento en espacio mundo.

dot(N, L) es el producto escalar, **max** evita valores negativos, **normalize** asegura vectores unitarios.

```
glUseProgram(shaderProgram); //

//El color de la luz
unsigned int lightColorLoc = glGetUniformLocation(shaderProgram, "lightColor");
glUniform3f(lightColorLoc, 0.9f, 0.6f, 0.6f);

// la posicion de la luz lo hago abajo poque va con el objeto de que se mueve
float lightPosX = 0.0 + -5 * sin((float)glfwGetTime() / 2.0f);
float lightPosY = 0.0;
float lightPosZ = 0.0 + (-5) * cos((float)glfwGetTime() / 2.0f);

unsigned int lightPosLoc = glGetUniformLocation(shaderProgram, "lightPos");
glUniform3f(lightPosLoc, lightPosX, lightPosY, lightPosZ);
```

- <u>Luz especular (Specular Lighting)</u>

Simula los reflejos brillantes y depende de la dirección del observador.

Luz Specular. Fragment.Shader

```
//la posicion del usuario/camara (0,0,10) en nuestro caso
unsigned int viewPosLoc = glGetUniformLocation(shaderProgram, "viewPos");
glUniform3f(viewPosLoc, 0.0f, 0.0f, 10.0f);
```

$$I = I_L \cdot K_S \cos^n(\alpha) = I_L \cdot K_S \cdot \left(\vec{R} \bullet \vec{V}\right)^n$$

```
#version 330 core
out vec4 FragColor;

in vec3 Normal;
in vec3 FragPos;

uniform vec3 viewPos;
uniform vec3 lightPos;
uniform vec3 lightColor;
uniform vec3 objectColor;
void main()
{
    // ambient
    float ambientI = 0.5;
    vec3 ambient = ambientI * lightColor;

    // diffuse
    vec3 norm = normalize(Normal);
    vec3 lightDir = normalize(lightPos - FragPos);
    float diff = max(dot(norm, lightDir), 0.0);
    vec3 diffuse = diff * lightColor;

          //Speculas
          float specularStrength = 1.0;
          vec3 viewDir = normalize(viewPos - FragPos);            //Usuario Fragmento
          vec3 reflectDir = reflect(-lightDir, norm);             //Reflejo de la luz
          float spec = pow(max(dot(viewDir, reflectDir), 0.0), 128); //Potencia
          vec3 specular = specularStrength * spec * lightColor;

          vec3 result = (ambient + diffuse + specular) * objectColor;
          FragColor = vec4(result, 1.0);
```

reflect permite calcular el vector reflejado, **pow** controla el brillo.